

E1-UDP02 — Verificação e análise

Evidência de verificação e análise das secções 4.1, 4.3 e 4.4 da ficha E1-UDP02. Os testes usam o servidor e o cliente já implementados (issues #3 e #5), sem alterações ao código.

Nota: os logs abaixo foram obtidos numa execução de referência do código atual. Antes de entregar, repetir os passos da secção 1 e confirmar/substituir os logs pelos da vossa execução.

1. Como reproduzir

Cliente e servidor são projetos independentes e correm em dois terminais separados.

Terminal 1 — servidor

```
cd server
javac UDPServer.java
java UDPServer
```

Saída esperada: **Servidor UDP a escutar no porto 6789.**

O estado do servidor (L, lista de receção e estrutura temporária) só existe em memória. **Reiniciar o servidor antes de cada cenário**, para começar com L = 0 e as estruturas vazias.

Terminal 2 — cliente

```
cd client
javac UDPClient.java
java UDPClient      # por omissão: localhost 6789
# ou: java UDPClient <host> <porto>
```

No arranque, o cliente pergunta o modo de numeração:

- **a** — automático: o cliente numera sozinho 1, 2, 3, ...
- **m** — manual: para cada mensagem, o cliente pede primeiro o texto e depois o número de sequência.

A desordenação é provocada com o modo manual, escolhendo nós os números. Para terminar o cliente, escrever **sair**.

2. Cenário normal (1, 2, 3 — modo automático)

Passos: reiniciar o servidor; no cliente, escolher **a** e escrever **olá, mundo, teste, sair**.

Respostas no cliente

Echo: 1,olá
Echo: 2,mundo
Echo: 3,teste

Log do servidor

Recebido: 1,olá
Resposta: 1,olá
L = 1
Temporárias: {}
Entregues neste passo: [olá]
Lista de receção: [olá]

Recebido: 2,mundo
Resposta: 2,mundo
L = 2
Temporárias: {}
Entregues neste passo: [mundo]
Lista de receção: [olá, mundo]

Recebido: 3,teste
Resposta: 3,teste
L = 3
Temporárias: {}
Entregues neste passo: [teste]
Lista de receção: [olá, mundo, teste]

Conclusão: com as mensagens em ordem, todas são entregues à chegada e a estrutura temporária mantém-se sempre vazia.

3. Cenário de desordenação múltipla (1, 3, 4, 2, 3 — modo manual)

Passos: reiniciar o servidor; no cliente, escolher **m** e introduzir:

Texto	Número
olá	1
mundo	3
tudo bem	4
cruel	2
mundo	3

e no fim **sair**.

Respostas no cliente

Echo: 1,olá
Servidor pede a mensagem em falta: waitingfor,2
Servidor pede a mensagem em falta: waitingfor,2
Echo: 2,cruel
Servidor pede a mensagem em falta: waitingfor,5

Log do servidor

Recebido: 1,olá
Resposta: 1,olá
L = 1
Temporárias: {}
Entregues neste passo: [olá]
Lista de receção: [olá]

Recebido: 3,mundo
Resposta: waitingfor,2
L = 1
Temporárias: {3=mundo}
Entregues neste passo: []
Lista de receção: [olá]

Recebido: 4,tudo bem
Resposta: waitingfor,2
L = 1
Temporárias: {3=mundo, 4=tudo bem}
Entregues neste passo: []

Lista de receção: [olá]

Recebido: 2,cruel

Resposta: 2,cruel

L = 4

Temporárias: {}

Entregues neste passo: [cruel, mundo, tudo bem]

Lista de receção: [olá, cruel, mundo, tudo bem]

Recebido: 3,mundo

Resposta: waitingfor,5

L = 4

Temporárias: {}

Entregues neste passo: []

Lista de receção: [olá, cruel, mundo, tudo bem]

3.1 Tabela

Mensagem enviada	Resposta recebida	L após processamento	Estrutura temporária	Mensagens entregues neste passo	Justificação
1, olá	1,olá (echo)	1	{}	olá	$L = 0$ e $N = 1 = L+1$: é a mensagem esperada e é entregue logo.
3, mundo	waitingfor ,2	1	{3=mundo}	—	$N = 3 \neq L+1 = 2$: chegou adiantada. É recebida mas não entregue; fica guardada. L não muda.
4, tudo bem	waitingfor ,2	1	{3=mundo, 4=tudo bem}	—	$N = 4 \neq 2$: também adiantada, também guardada. Continua a faltar a 2.

2, cruel	2, cruel (echo)	4	{}	cruel, mundo, tudo bem	N = 2 = L+1: é entregue e desencadeia a entrega em cascata da 3 e da 4, que estavam guardadas. São entregues três mensagens neste passo, L salta de 1 para 4 e a estrutura temporária fica vazia.
3, mundo	waitingfor ,5	4	{}	—	Duplicado de uma mensagem já entregue (N = 3 ≤ L = 4). Não é entregue outra vez nem vai para a estrutura temporária; o estado não muda. O servidor responde waitingfor,5 , a mesma resposta que daria a uma mensagem adiantada (ver 5.4).

3.2 Estado final

- **Lista de receção:** [olá, cruel, mundo, tudo bem]. Tem as quatro mensagens distintas, **por ordem de número (1, 2, 3, 4)**, apesar de terem chegado pela ordem 1, 3, 4, 2. O duplicado final não aparece repetido.
- **Estrutura temporária:** {}. Está vazia porque todas as mensagens que chegaram adiantadas (3 e 4) foram entregues na cascata desencadeada pela 2, e cada uma foi retirada da estrutura temporária no momento em que foi entregue. O duplicado final não foi lá colocado.

Onde foi construída a ordem: no servidor, no método `processDeliveredMessages`. A estrutura temporária guarda cada mensagem pelo seu número, e o ciclo de entrega só passa para a lista de receção a mensagem L+1, uma de cada vez. A rede entregou as mensagens desordenadas; foi a aplicação que as ordenou.

4. Custo da retransmissão: UDP001 vs UDP02 (secções 4.1 e 4.4)

4.1 Premissas

1. O cliente envia as mensagens sem que o envio de cada uma dependa da resposta à anterior. A ordem indicada é a **ordem de chegada** ao servidor.
2. Conta-se como retransmissão cada mensagem que já tinha chegado ao servidor, ou se perdeu, e tem de voltar a ser enviada.
3. As retransmissões chegam por ordem, sem novas perdas.
4. Nas sequências 1, 3, 4, 5, 2 e 1, 3, 4, 5, 6, 2, a mensagem 2 **não se perde**: chega atrasada, em último lugar.
5. No exemplo da perda, a mensagem 2 **perde-se** e tem de ser reenviada nas duas versões. Nenhuma das versões deteta a perda nem reenvia sozinha; o reenvio é feito pelo utilizador.

4.2 Contagens

Sequência 1, 3, 4, 5, 2

Versão	O que acontece	Retransmissões
UDP001	3, 4 e 5 chegam antes da 2, recebem waitingfor, 2 e são descartadas . Depois de a 2 ser aceite, têm de ser reenviadas.	3 (3, 4, 5)
UDP02	3, 4 e 5 ficam na estrutura temporária. Quando chega a 2, são entregues 2, 3, 4 e 5 em cascata.	0

Sequência 1, 3, 4, 5, 6, 2

Versão	O que acontece	Retransmissões
UDP001	3, 4, 5 e 6 são descartadas e têm de ser reenviadas.	4 (3, 4, 5, 6)
UDP02	3, 4, 5 e 6 ficam guardadas e são entregues em cascata quando chega a 2.	0

Perda da mensagem 2 numa sequência de 10 (1 ... 10)

Versão	O que acontece	Retransmissões
--------	----------------	----------------

UDP001	A 2 perde-se. 3 a 10 chegam, são todas descartadas com <code>waitingfor, 2</code> . É preciso reenviar a 2 e depois 3 a 10.	9 (2, 3, ..., 10)
UDP02	A 2 perde-se. 3 a 10 ficam guardadas. Basta reenviar a 2, cuja chegada desencadeia a cascata de 2 a 10.	1 (2)

4.3 Porque é que a diferença existe

Na UDP001, uma mensagem que chega adiantada é deitada fora, **mesmo já tendo atravessado a rede com sucesso**. Um único atraso ou perda obriga a reenviar todas as mensagens que chegaram depois dele. Com a mensagem em falta na posição 2 de uma sequência de k mensagens, isso custa k – 2 reenvios num atraso, ou k – 1 numa perda.

Na UDP02, essas mensagens são **aproveitadas**: ficam guardadas na estrutura temporária como recebidas, e são entregues assim que a sequência fica completa. Só é preciso reenviar a mensagem que realmente falta.

5. Reflexão crítica (secção 4.4)

5.1 Mensagem recebida vs mensagem entregue

- Uma mensagem é **recebida** quando o datagrama chega ao servidor.
- Uma mensagem é **entregue** quando passa para a lista de receção, o que só acontece quando todas as anteriores também já foram entregues.

No cenário da secção 3, a mensagem 3 foi recebida no segundo datagrama mas só foi entregue no quarto. Entre esses dois momentos esteve na estrutura temporária. É lá que a diferença fica visível no servidor.

5.2 Mensagem em falta que nunca chega

Se a mensagem L+1 nunca chegar:

- a cascata nunca é desencadeada;
- nenhuma mensagem à frente do buraco volta a ser entregue;
- as mensagens guardadas ficam na estrutura temporária indefinidamente;
- todas as mensagens seguintes recebem `waitingfor, <L+1>`.

A solução não tem nenhuma noção de tempo. Faltaria um mecanismo baseado em **temporizadores**:

- o servidor registaria há quanto tempo espera por L+1 e, passado um limite, voltaria a pedir essa mensagem ao cliente;
- e/ou o cliente usaria um timeout na receção e reenviaria as mensagens sem resposta.

Nota: o cliente atual não tem timeout. Se um datagrama ou a sua resposta se perder, fica bloqueado no `receive()`.

5.3 Crescimento da estrutura temporária / mensagem muito adiantada

Sem nenhum limite, uma mensagem com número 1 000 000 seria guardada na estrutura temporária e nunca seria entregue, porque faltariam 999 999 mensagens antes dela. Um cliente malicioso, ou com um erro, poderia enviar muitos números diferentes e fazer crescer a estrutura até esgotar a memória do servidor.

No código atual, o crescimento é **controlado pelo servidor**: um N mais de 10 000 à frente de L (`MAX_SEQUENCE_AHEAD`) é rejeitado com `error,invalid_datagram` e não é guardado. Isto limita a estrutura temporária a cerca de 10 000 entradas.

Execução de verificação (servidor reiniciado; sequência 1, 3, 3, 2, 1000000):

Datagrama inválido (número demasiado distante da sequência atual): 1000000, longe

Resposta: error,invalid_datagram

L = 3

Temporárias: {}

Entregues neste passo: []

Lista de receção: [olá, cruel, mundo]

O limite reduz o risco mas não o elimina: um cliente pode ocupar até ~10 000 entradas de até 1000 bytes cada. Além disso, o limite é partilhado por todos os clientes (ver 5.5).

5.4 Duplicados

Duplicado de uma mensagem já entregue ($N \leq L$). No teste da secção 3, o segundo 3,mundo chegou com L = 4.

- **Comportamento observado:** não foi entregue outra vez, não foi colocado na estrutura temporária, e L e a lista de receção não mudaram. A resposta foi `waitingfor,5`.
- **Porque acontece:** o método `processDeliveredMessages` devolve L sem alterações quando $N \leq L$.
- **Limitação:** o duplicado é descartado, mas **não é identificado como duplicado**. O cliente recebe a mesma resposta que receberia por uma mensagem adiantada e não consegue distinguir os dois casos. Também não se verifica se o conteúdo é igual ao da mensagem original com o mesmo número.

Duplicado de uma mensagem ainda na estrutura temporária. Execução de verificação (sequência 1, 3, 3, 2):

Recebido: 3,mundo

Resposta: waitingfor,2

L = 1

Temporárias: {3=mundo}

Recebido: 3,mundo

Resposta: waitingfor,2

L = 1

Temporárias: {3=mundo}

Recebido: 2,cruel

Resposta: 2,cruel

L = 3

Temporárias: {}

Entregues neste passo: [cruel, mundo]

Lista de receção: [olá, cruel, mundo]

- **Comportamento observado:** a estrutura temporária continua com uma única entrada para a mensagem 3, porque é indexada pelo número e o código usa `putIfAbsent`, que mantém a primeira cópia. Na cascata, a 3 é entregue uma única vez.
- **Limitação:** tal como no caso anterior, o duplicado não é assinalado e a resposta é igual à de uma mensagem adiantada. Se o segundo datagrama tivesse o mesmo número mas conteúdo diferente, seria mantido o primeiro sem aviso.

Conclusão: os duplicados observados não causaram entregas repetidas. Mas a solução **não deteta nem sinaliza duplicados**: limita-se a não os entregar, e a resposta ao cliente não os distingue de mensagens fora de ordem.

5.5 Vários clientes

L, a lista de receção e a estrutura temporária são únicas no servidor, e a numeração é feita por cada cliente. Com dois clientes em simultâneo, as numerações misturam-se: a mensagem 1 do segundo cliente seria tratada como um duplicado da mensagem 1 do primeiro. O estado só faz sentido para um cliente. Seria necessário um conjunto de estruturas (L, lista, temporária) por cliente, identificado pelo endereço e porto de origem do datagrama.

5.6 Características do UDP: resolvidas e por resolver

Característica	Situação
Desordenação o	Resolvida. As mensagens são reordenadas no servidor sem ser preciso retransmitir as que chegaram adiantadas (secções 3 e 4).
Duplicação	Parcialmente. Os duplicados observados não foram entregues duas vezes, mas não são detetados nem sinalizados ao cliente (5.4).

Perda	Não resolvida. Não há detecção de perdas por tempo nem retransmissão automática. Uma perda definitiva bloqueia a entrega e deixa mensagens retidas indefinidamente (5.2).
Corrupção	Não resolvida pelo mecanismo. Os datagramas corrompidos são detetados e descartados pelo checksum do UDP antes de chegarem à aplicação, o que, para a aplicação, equivale a uma perda. O servidor também rejeita datagramas que não são UTF-8 válido, mas não recupera o conteúdo.

6. Porque é que a desordenação foi provocada manualmente

Em localhost, cliente e servidor estão na mesma máquina. Os datagramas não atravessam routers, caminhos alternativos nem filas com atrasos diferentes, que são as causas da desordenação numa rede real. Seguem todos o mesmo percurso, quase instantâneo, e praticamente nunca chegam trocados. Para verificar o mecanismo foi por isso necessário **simular** a ordem de chegada com o modo de numeração manual do cliente, escolhendo os números de sequência.